

Build Your Own AI Chatbot SaaS

The Practical Guide — start a real business with free tools, explained like you're 18

No coding background? Good — you don't need one. Over four relaxed weeks you'll set up a robot that answers questions, books appointments, and sends emails on any business's website, then sell it as a monthly subscription. Cost to you: \$0 and about 8 focused hours. Every step has a checkbox and a "how to know it worked" test.

Audience — non-technical future business owner **Version** — v1.0.0

Date — September 2026 **Reading time** — ~45 min

Total effort — ~8 hours over 4 weeks

WHAT'S INSIDE

The robot-helper idea

The 6 free tools

Zero-jargon how-it-works

Week-by-week setup

Teaching the brain

Bookings & the widget

Launch & first invoice

Running the business

Top-20 fixes

Quick-reference cards

All six tools are free forever · no credit card · the only investment is a few evenings of your time

Contents

All 10 chapters. Every entry is a link — click to jump, in the PDF or in your browser. Page numbers are the printed page numbers.

1. The Big Picture	4
2. The 6 Free Tools	7
3. How It Works	10
4. Week 1: Foundation (Day by Day)	12
5. Week 2: Teaching the Brain	17
6. Week 3: Appointments & the Widget	20
7. Week 4: Your Dashboard & Launch	23
8. Running the Business	27
9. Troubleshooting — Top 20	30
10. Quick Reference Cards	33

⚠ **STACK UPDATE (SEPTEMBER 2026) — READ THIS FIRST.** This guide was written when the backend ran as a **DOCKER SPACE ON HUGGING FACE**. HF removed the Docker-SDK free tier in July 2026 (only Static Spaces are free), so the codebase now ships a **CLOUDFLARE WORKER** backend (`worker/`) with **GOOGLE GEMINI** for chat + embeddings. Before starting, follow `DEPLOY.md` / `SETUP.md` for the current steps and use the Worker URL `https://chatbot-api.<subdomain>.workers.dev` wherever a chapter says `...hf.space` . Chapters about later features (appointments, emails, admin dashboard, PDF upload) are the **ROADMAP** — they land in Phases 2–4, described in the Appendix “Migration Roadmap” at the end of this guide. The old Python backend is kept in `backend/` as the reference implementation.

1. The Big Picture (5 min read)

You're going to build a **robot helper** that lives on websites. Picture a little robot that pops up in the corner of a business's website and says *"Hi! How can I help you?"* It does four things for that business, 24 hours a day, 7 days a week:

-  **Chats with visitors** — answers questions instantly, even at 2 AM
-  **Books appointments** — a visitor says "Book Tuesday at 2pm" and it actually books it
-  **Answers from the business's own info** — you upload their FAQ PDF and the robot answers from *that*, not from thin air
-  **Sends confirmation emails** — bookings trigger automatic confirmation emails (Chapter 6 explains the one caveat about how the customer's real address is captured)

And here's the business model: you sell this to local businesses — dentists, hair salons, plumbers, tutors, consultants, auto shops. They pay you a monthly subscription. You manage every client from one control panel. One robot brain, many clients, each with their own name, colors, and info. That's what "SaaS" means: **Software as a Service** — one software, many customers paying monthly.

Why this can actually work

Cost to you	Cost to your client	What they get
\$0 — every tool is free, no credit card	\$49–\$199/month (you pick)	Never miss a lead again, bookings without a receptionist, instant answers 24/7
~8 hours to launch your first client	~30 seconds to install (one line of code)	Looks professional, works on their phone, no hardware
~2 hours/week to run it	—	—

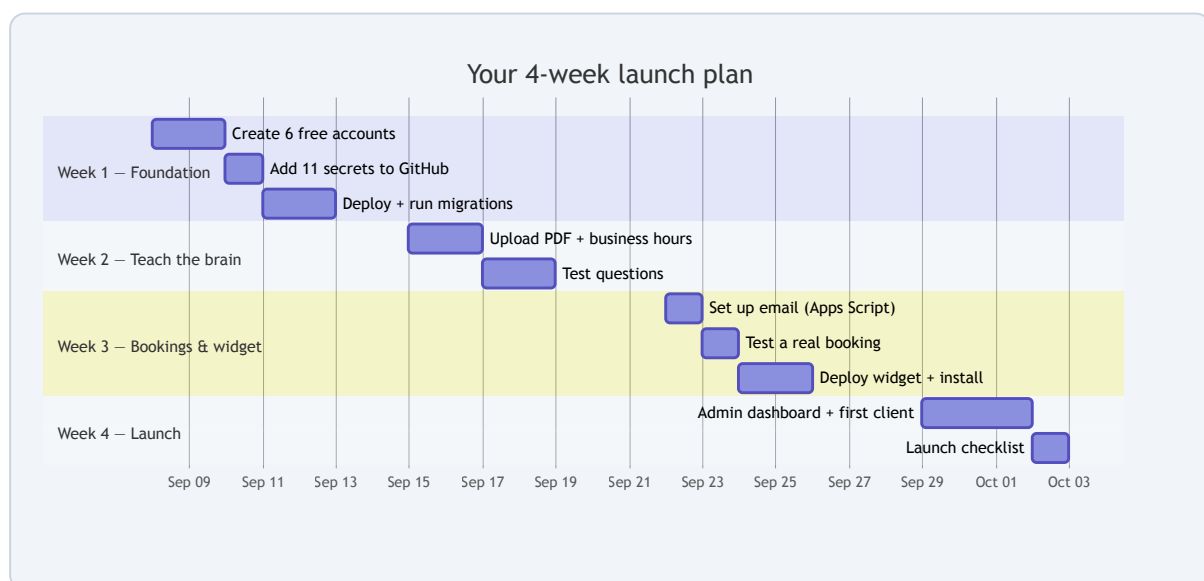
What the money could look like

Clients	Price/client	Monthly revenue	Time per week
1 (pilot)	\$49	\$49	1–2 h
5	\$79	\$395	3–4 h
15	\$99	\$1,485	6–8 h (time to think about a VA)
30	\$129	\$3,870	Full-time job (hire help)

These are realistic numbers for local businesses in most cities — not promises. The point is the math works even at the low end: 5 clients at \$79 covers a lot of pizza.

HONEST EXPECTATIONS. Your first client will take longer than 8 hours, because you'll hit bumps nobody can predict. Your second takes half that. Your tenth takes 15 minutes. The product is the *system*, and the system gets smoother every week. Don't quit after client #1's hiccups.

What you'll actually do over the next month



Total time: ~6–8 focused hours over 4 weeks. **Total money:** \$0. **What you need:** a laptop, an email address you actually check, and one local business owner who likes you. That last part is the hardest — and Chapter 8 is all about it.

2. The 6 Free Tools (10 min read)

You need exactly **6 accounts**. All free, all with a "free forever" tier, **none require a credit card**. Here's what each one does, in plain English, plus exactly what to click when you sign up.

Tool	Website	In one sentence	What it costs
GitHub	github.com	The warehouse where your project's code lives, and the robot that deploys it (GitHub Actions)	Free
Cloudflare	cloudflare.com	Runs the AI backend for free — a Worker (serverless code, never sleeps) + Pages (static hosting for the widget and dashboard)	Free
Google AI Studio	aistudio.google.com	Gemini API — the actual AI brain (chat + embeddings) on a generous free tier, no card needed	Free
Turso	turso.tech	The filing cabinet — a database that stores every chat, booking, and client	Free (9 GB, 1 B reads/mo)
Cloudflare	cloudflare.com	The delivery service — hosts your chat widget file and your admin dashboard, with unlimited bandwidth	Free
Clerk	clerk.com	The bouncer — handles "who is allowed into your admin dashboard" logins for you	Free (10k users/mo)
Google Apps Script	script.google.com	A free robot inside your Gmail that sends the booking emails	Free (100 emails/day)

Why these 6 and not others?

- **GitHub** is the only one that stores code AND deploys it automatically. Without it you'd be uploading files by hand like it's 1999.
- **Cloudflare Workers** are the free host for the AI backend now. Hugging Face used to host it (Docker Spaces) but moved that behind PRO in July 2026 — so the backend is a TypeScript Worker that calls the Gemini API. No GPU, no model downloads, nothing to keep awake.
- **Turso** gives you a real database with vector search built in, free, with no server to manage. Postgres hosts start charging for what Turso gives away.
- **Cloudflare** has genuinely unlimited bandwidth on free Pages — your widget can go viral and you pay nothing.
- **Clerk** does authentication better than you ever will — passwords, magic links, session security. Hand-rolling logins is how sites get hacked.
- **Google Apps Script** turns your existing Gmail into an email API. Every email service (SendGrid, Resend) needs a credit card and a verified domain; Gmail is already verified.

Signup checklist — what to click

☐ **GitHub** → github.com → "Sign up" → email + password → verify the email they send you.

☐ **Cloudflare** → cloudflare.com → "Sign Up" → My Profile → **API Tokens** → **Create Token** → template "Edit Cloudflare Workers" (+ add Account → Cloudflare Pages:Edit) → copy the token; note your **Account ID** from the dashboard. You'll paste both into GitHub later.

☐ **Google AI Studio** → aistudio.google.com/apikey → "Create API key" → copy it (GEMINI_API_KEY). This powers the AI brain now that Docker-on-HF is paid.

☐ **Turso** → turso.tech → "Sign up" with GitHub → click **Create Database** → name it chatbot-db → pick a region near you → create → click the database → **Show**

connection details → copy the **URL** (starts with `libsql://`) and the **auth token**. Save both.

- ☐ **Clerk** → clerk.com → sign up with GitHub → create an application, name it Chatbot Admin → in the dashboard go to **API Keys** → copy **Publishable Key** (starts `pk_test_`) and **Secret Key** (starts `sk_test_`). Save both. Then: **User & Authentication** → **Email, Phone, Username** → enable **Email** only → add your own email to the **Allowlist**.

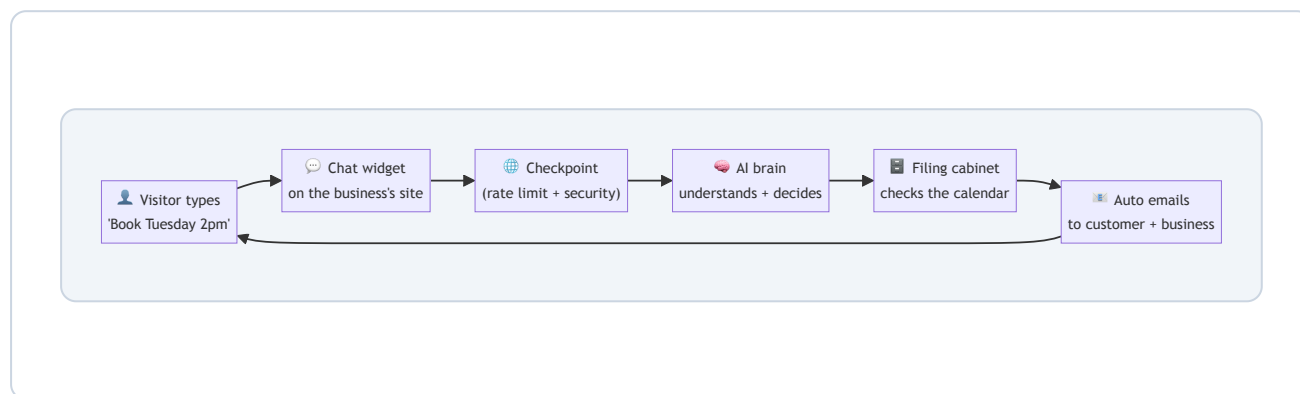
- ☐ **Cloudflare** → cloudflare.com → sign up → after login, look at the dashboard's right sidebar for your **Account ID** (long hex string). Save it. Then: profile icon → **My Profile** → **API Tokens** → **Create Token** → pick the "**Edit Cloudflare Workers**" template → on the permissions page add **Account** → **Cloudflare Pages** → **Edit** (the Workers template alone doesn't cover Pages, which is what deploys your widget and admin) → continue → create → copy the token (shows once!). Save it.

- ☐ **Google Apps Script** → script.google.com → "New Project" → name it Chatbot Email Sender → delete the default code → paste the contents of `gas-email/Code.gs` from the project → click **Deploy** → **New deployment** → type gear → **Web app** → Execute as: **Me** → Who has access: **Anyone** → Deploy → copy the URL (ends in `/exec`). Save it. Then run the `testEmail` function once (it's in the code) and click through the Google permission screen — this "authorizes" the robot to send email as you.

KEEP A NOTES FILE OPEN. You're collecting 8+ secret values during this chapter (tokens, URLs, keys). Put them in a private notes file *as you go* — Chapter 4 needs them all in one place. Never paste them into a public chat or repo.

3. How It Works (3 min read)

Zero jargon version. When a visitor types a message on a client's website, here's the whole journey — under 3 seconds:



1. **The visitor types.** The chat bubble on the website captures the message.
2. **The checkpoint.** The message passes through a security gate (nginx on the AI's host) that limits how fast anyone can spam and checks the request is legit.
3. **The AI brain understands.** A small AI model figures out what the visitor wants — booking, a question, or a human. It can do this because it was "shown" examples during setup.
4. **The filing cabinet checks.** For questions, it searches the business's uploaded PDF by *meaning*, not keywords. For bookings, it checks the business hours and existing appointments and picks open slots.
5. **Emails fly out.** Google Apps Script sends the confirmation to the customer and a heads-up to the business.

The whole thing costs **\$0/month** because: the AI runs on the Gemini API free tier (called by a Cloudflare Worker — Hugging Face's Docker hosting is no longer free, so the backend moved to Cloudflare), the data sits in Turso's free database, the widget and dashboard are served by Cloudflare Pages' free network, the emails go through your free Gmail, and the logins use Clerk's free tier. The only thing you pay is attention.

RESTAURANT ANALOGY. The widget is the waiter taking orders. The AI brain is the chef who decides what dish to cook. The database is the reservation book and recipe box. Apps Script is the mailroom. You're the restaurant owner — you set the menu (upload the PDF), the hours, and the prices. The customers never see the kitchen.

4. Week 1: Foundation (Day by Day)

This week is all accounts, secrets, and deploys. It's the most clicky week — **2–3 hours total**. Work through it in order; every day ends in a test you can see pass, so you always know it worked.

Day 1 — Create the 6 accounts (45 min)

Exactly what to click is in Chapter 2. By the end of Day 1 you should have:

- ☐ GitHub account, logged in
- ☐ Hugging Face account + `HF_TOKEN` (starts `hf_`) saved
- ☐ Turso database `chatbot-db` + URL + token saved
- ☐ Clerk app + `pk_test_...` and `sk_test_...` saved, your email allowlisted
- ☐ Cloudflare Account ID + API token saved
- ☐ Google Apps Script project with `Code.gs` pasted, deployed as web app, URL saved, `testEmail()` run once (check your Gmail!)

How to know it worked: you have 8 saved values in your notes file and a test email in your inbox.

Day 2 — Get the code and add the 11 secrets (30 min)

You need a copy of this project on GitHub under *your* account. If you have a template repo link, click **Use this template** → name it (e.g. `my-chatbot-saas`) → create. If you already have the repo, skip.

Now add the secrets. On your GitHub repo page: **Settings → Secrets and variables → Actions → New repository secret**. Add these 11, one by one (copy each from your notes file):

Secret name (exact)	Value from
TURSO_DATABASE_URL	Turso connection details (starts <code>libsql://</code>)
TURSO_AUTH_TOKEN	Turso connection details
CLERK_PUBLISHABLE_KEY	Clerk → API Keys (starts <code>pk_test_</code>)
CLERK_SECRET_KEY	Clerk → API Keys (starts <code>sk_test_</code>)
GAS_WEBAPP_URL	Apps Script deploy URL (ends <code>/exec</code>)
GEMINI_API_KEY	Google AI Studio key (replaces the old local model — chat + embeddings run on Gemini)
ADMIN_EMAILS	Your email(s), comma-separated — allowed on <code>/admin/*</code> (must match your Clerk allowlist)
ADMIN_BOOTSTRAP_TOKEN	Optional long random string — only for the first tenant/knowledge uploads (delete once the dashboard works, Phase 3)
CLOUDFLARE_API_TOKEN	Cloudflare API token
CLOUDFLARE_ACCOUNT_ID	Cloudflare dashboard → Account ID
ADMIN_EMAIL	Your email (must match your Clerk allowlist)

How to know it worked: the secrets page lists all the names with no typos. Secrets are hidden after saving — that's normal. If you typo one, delete and re-add it.

Day 3 — Deploy the AI backend to Cloudflare Workers (30 min)

1. Set the GitHub secrets from Chapter 2 — especially `TURSO_DATABASE_URL` , `TURSO_AUTH_TOKEN` , `GEMINI_API_KEY` , `CLOUDFLARE_API_TOKEN` ,

`CLOUDFLARE_ACCOUNT_ID` . (No Hugging Face accounts or Space names needed anymore.)

2. Create the Turso database if you haven't: `turso db create chatbot-db` .
3. Push anything to `main` in your GitHub repo (or run the *Deploy Chatbot API Worker to Cloudflare* workflow manually: **Actions** → **Deploy Chatbot API Worker** → **Run workflow**). The workflow typechecks `worker/` , deploys with `wrangler` , and writes the Worker secrets.
4. Wait ~1–3 minutes (no AI models to download — the model lives in the Gemini API). The Actions tab shows progress.

How to know it worked: open `https://chatbot-api.YOUR-SUBDOMAIN.workers.dev/health` in a browser (run `npx wrangler whoami` locally to see your subdomain). You should see `{"status":"healthy","version":"1.0.0",...}` . Workers don't sleep, so no keep-alive is needed — but watch Gemini's daily free quota (Chapter 9 troubleshooting).

WHERE THINGS STAND (SEPTEMBER 2026). The repo you're cloning is mid-migration from the old Hugging Face Docker backend to the Cloudflare Worker stack. What's implemented now (Phase 1): widget chat with RAG answers, knowledge upload (text/FAQ), conversation history, and Worker deploys via GitHub Actions. What's still being ported (Phases 2–4): appointment booking + emails, the full admin dashboard, PDF upload, and the legacy `scripts/` . The old Python backend lives on in `backend/` as the reference implementation. The Appendix at the end of this guide spells out the phases in detail.

Day 4 — Set up the database (15 min)

1. **Create the database** (one time): `turso db create chatbot-db` .
2. **Load the schema** — the Worker needs the tables to exist before anything works (idempotent — safe to re-run):

```
turso db shell chatbot-db < worker/db/schema.sql
```

This creates all 8 tables including the vector table `knowledge_vec` (sqlite-vec is built into Turso). No model downloads — embeddings run on the Gemini API.

3. Copy the connection URL + auth token (`turso db show chatbot-db`) into the GitHub secrets `TURSO_DATABASE_URL` / `TURSO_AUTH_TOKEN` , then re-run the Day 3 deploy workflow.

How to know it worked: in Turso's dashboard → your database → **Explore/SQL**, run `SELECT name FROM sqlite_master WHERE type='table';` — you should see `tenants` , `conversations` , `messages` , `appointments` , `knowledge_chunks` , `knowledge_vec` , `usage_logs` , `end_users` .

Day 5 — Deploy the widget and admin (25 min)

1. Cloudflare dashboard → **Workers & Pages** → **Create** → **Pages** → **Connect to Git** → pick your repo.
2. Project name: `chatbot-widget` → Production branch: `main` → Framework: none → Build command: `cd widget && npm run build` → Build output directory: `widget/dist` → Save and Deploy.
3. Repeat for `chatbot-admin` with build command `cd admin-dashboard && npm run build` and output `admin-dashboard/dist` .
4. **Give the admin build your Clerk key.** The admin app bakes its Clerk publishable key in at *build time*. In the `chatbot-admin` Pages project go to **Settings** → **Environment variables** → add `VITE_CLERK_PUBLISHABLE_KEY` = the `pk_test_...` value from your notes → Save → redeploy. (If you deploy via GitHub Actions instead of the Pages git integration, add the same variable to the `npm run build` step of `.github/workflows/deploy-admin.yml` .) Without it the admin page loads but the login screen never appears.
5. Then push to `main` (or trigger the workflows) to build them via GitHub Actions instead — pick one path (Pages git integration or Actions) to avoid double deploys.

How to know it worked: visit `https://chatbot-widget.pages.dev/widget.js` — you should download a JS file (the build emits exactly this filename). Visit `https://chatbot-admin.pages.dev` — you should see a login screen (Clerk). The dashboard is fully wired once the key above is set: tenant CRUD, conversations, appointments, knowledge upload, and settings save all go through the admin API with your session token attached automatically.

WEEK 1 DONE = SKELETON STANDING. You now have: a live AI brain with a database, a downloadable widget, and a login screen. Nothing is "yours" yet — Week 2 teaches the brain.

5. Week 2: Teaching the Brain

Now the robot learns its first real job. You'll feed it a client's information (a PDF), let it read the business hours, and test that it answers correctly. ~1 hour this week.

Day 1 — Create your first tenant (15 min)

A *tenant* is a client — one business with its own robot personality. In the Space terminal (you did the Day-3 Dockerfile extra from Week 1, right? These commands need it):

```
python scripts/create_tenant.py --name "Weblyft Design" --slug weblyft-design
--color "#3B82F6"
```

`--slug` is the robot's address: lowercase letters, numbers, and hyphens only. It becomes part of the install code, so pick something clean (`weblyft-design` , not `Weblyft Design!!!`).

How to know it worked: the script prints the tenant's ID and a ready-made embed code. Or check Turso: `SELECT slug, name FROM tenants;`

Day 2 — Upload the client's brain food (15 min)

Ask your client for their FAQ, services list, price list, or about page — anything with facts they want the robot to know. Put it in a PDF (or plain text). Then:

```
python scripts/upload_knowledge.py --tenant weblyft-design --file faq.pdf
```

What happens inside (plain version): the PDF is read, chopped into ~500-word bite-size pieces, each piece is turned into a "meaning fingerprint" (a list of 768 numbers — an *embedding*), and all of it is filed in the database. Later, when a visitor asks something, the robot fingerprints their question too and pulls the pieces whose fingerprints are most similar.

How to know it worked: the script prints `Created N chunks from M characters` . In Turso: `SELECT COUNT(*) FROM knowledge_chunks WHERE tenant_id = (SELECT id FROM tenants WHERE slug='weblyft-design');`

Day 3 — Teach it the business hours (10 min)

```
python scripts/extract_business_hours.py --tenant weblyft-design --file
faq.pdf
```

The AI reads the PDF itself, finds the opening hours, and saves them as the client's calendar rules. (If your PDF has no hours, skip this — the robot defaults to Mon–Fri 9–5, which you can change later in the dashboard.)

How to know it worked: the script prints the hours JSON it found. Verify in Turso: `SELECT business_hours FROM tenants WHERE slug='weblyft-design';`

Day 4 — Test questions (20 min)

Test the robot exactly like a customer would. In the Space terminal (or any terminal with curl):

```
curl -X POST "https://chatbot-api.YOUR-
SUBDOMAIN.workers.dev/widget/chat/weblyft-design" \
-H "Content-Type: application/json" \
-d '{"message": "What are your hours?", "session_id": "test123"}'
```

You type	Expected behavior
"What are your hours?"	Answers from the uploaded PDF — the exact hours, not a guess
"Book an appointment for tomorrow 2pm"	Checks the calendar, confirms the booking, streams a confirmation
"What times are open Friday?"	Shows available slots

"Do you offer refunds?"	Answers from the PDF, or honestly says it doesn't know (never makes it up)
"Talk to a human"	Polite handoff message

How to know it worked: each answer makes sense and the streamed text arrives in pieces. If an answer is wrong, re-check the PDF's wording — the robot can only know what the PDF says.

Day 5 — Polish (15 min)

- Try weird phrasings: "can I come in on saturday?", "book me in" — the robot should still route them correctly (it was trained on examples like this).
- If something keeps misclassifying (e.g. "book" triggers a general answer), add the exact phrase to the few-shot examples in `backend/app/services/llm.py` and redeploy — the deep-dive guide explains how.
- Re-upload a cleaner PDF if answers are vague. Better input = better robot.

THE "HOW TO KNOW IT WORKED" HABIT. Every step in this guide has a checkable result. When you're running a business on this, that habit is what saves you: change one thing, verify one thing, move on. Never change three things and hope.

6. Week 3: Appointments & the Widget

Week 3 makes the robot *visible and useful*: real booking emails, then the chat button on a real website. ~2 hours.

Day 1 — Make sure email actually sends (20 min)

You already deployed the email robot in Week 1. Now prove it end to end. Easiest: open your Apps Script project → run the `testEmail` function → check your inbox. If you did the Day-3 Dockerfile extra, you can also use the terminal version — in the Space terminal:

```
python scripts/test_gas_email.py --to your@email.com
```

How to know it worked: you get `✓ Email sent successfully!` and the test email lands in your inbox within seconds. If it fails, check: the `GAS_WEBAPP_URL` secret is the `/exec` URL, the deployment is "Anyone" + "Execute as me", and you ran `testEmail()` once in Apps Script to authorize.

Day 2 — Test a real booking (20 min)

Chat with your own robot like a customer:

```
curl -X POST "https://chatbot-api.YOUR-  
SUBDOMAIN.workers.dev/widget/chat/weblyft-design" \  
-H "Content-Type: application/json" \  
-d '{"message": "Book an appointment for tomorrow at 2pm", "session_id":  
"test-booking"}'
```

Then check Turso — an appointment row should exist:

```
SELECT start_time, end_time, status FROM appointments;
```

How to know it worked: the chat confirms the booking and a new row appears when you re-run the appointments query. Book a second appointment at the same time — the robot should refuse it and offer alternatives (that's the conflict check doing its job). On the email front, read the quirk right below: the chat path sends its confirmation to a placeholder address, so a real email only goes out when you use the dedicated booking endpoint with the visitor's actual email.

ONE KNOWN QUIRK IN THE CURRENT CODE. The chat-based booking sends its confirmation email to a placeholder address (`customer@example.com`), while the dedicated booking endpoint (`POST /widget/appointments/{slug}`) emails the real customer address you pass in the request. For production, capture the visitor's email in the widget (a tiny "what's your email?" step) and pass it through — the deep-dive guide flags this in Chapter 12.

Day 3 — Build and deploy the widget (20 min)

If you set up Cloudflare Pages in Week 1, the widget is already deployed. Get your widget URL:

```
https://chatbot-widget.pages.dev/widget.js
```

Test it on a scratch page — create any HTML file with this one line before `</body>` :

```
<script src="https://chatbot-widget.pages.dev/widget.js" data-tenant="weblyft-design"></script>
```

Open the file in a browser. **How to know it worked:** a floating chat button appears bottom-right, in the client's color, and clicking it opens the chat with the greeting you set.

Day 4 — Install it on the client's site (15 min)

Send the client one instruction — or better, do it with them on a call:

1. Open their website's HTML (WordPress: Appearance → Theme File Editor → `footer.php` ; or their developer pastes it in the site template).
2. Paste the one-line script tag just before the closing `</body>` tag.
3. Save and refresh the site.

How to know it worked: the button appears on their live site. Test on their phone too — the widget is responsive and the booking flow works on mobile (Chapter 6 of the deep-dive explains the mobile slot picker).

Day 5 — Pilot polish (30 min)

- Book from a phone browser (Safari/Chrome) — slots should be tappable.
- Test the "closed day" case: ask for Sunday — no slots should appear.
- Test the timezone case: if the business is in New York, make sure `timezone` is `America/New_York` on the tenant (dashboard → Settings tab, or a quick SQL update).
- Take a screenshot of the working widget on their site — you'll use it in your pitch for client #2.

WEEK 3 DONE = THE PRODUCT EXISTS. A business's website now has a 24/7 receptionist that books real appointments and emails confirmations. You have proof for the next sale.

7. Week 4: Your Dashboard & Launch

This week you take the wheel: the admin dashboard, your first real client, monitoring, and the launch checklist. ~1.5 hours.

Day 1 — Log in and look around (20 min)

Open `https://chatbot-admin.pages.dev` → sign in with Clerk (use the email you allowlisted — if you get "couldn't sign you in", add the admin URL to **Clerk → Domains** and allowed redirect URLs).

Tour the screens:

- **Tenants** — your client list. Click one to open its detail page.
- **Overview tab** — conversations, appointments, confirmed vs pending counts.
- **Conversations tab** — every chat thread with its last message.
- **Appointments tab** — every booking with date, time, status.
- **Knowledge tab** — upload a client's PDF/TXT from the browser (note: this page still needs its auth wiring, so if an upload fails, use the Week 2 terminal commands instead — they do the same job).
- **Settings tab** — the business-hours editor (timezone, slot length, buffer) — saving from here is still being wired up, so until then set hours with the Week 2 extraction command or the SQL in Chapter 10).
- **Analytics** — platform totals and your busiest clients.

HONEST STATUS OF THE DASHBOARD. Sign-in works once the Week-1, Day-5 key is set, but in the current code the *data* screens need two wiring steps that aren't finished: the admin app must attach your Clerk session token to its API calls, and its page layout must mount the routes. Until those land, lists can stay empty even right after a successful login — that's a known code gap, not something you did wrong. The terminal recipes from Weeks 2–3 (create tenant, upload PDF, extract hours, test booking) do exactly the same jobs and work today, so lean on them while the dashboard catches up.

Day 2 — Add your first real client (20 min)

The 5-minute client recipe, dashboard edition:

1. Tenants → **Add Tenant** → name (e.g. "Bob's Plumbing"), slug (`bobs-plumbing`) → Create. (Heads-up: the dashboard's color picker is buggy — the chosen color won't stick. Set the color from the terminal instead: `python scripts/create_tenant.py --name "Bob's Plumbing" --slug bobs-plumbing --color "#10B981"` if you're creating it that way, or patch it later with SQL.)
2. Upload their FAQ PDF with the Week 2 command (Day 2), using their slug: `python scripts/upload_knowledge.py --tenant bobs-plumbing --file faq.pdf`.
3. Set their timezone and hours in the dashboard (Tenant → Settings tab saves hours/timezone/slot duration/buffer directly), or re-run the extraction command (Week 2, Day 3) with their slug.
4. Copy the embed code (Week 3, Day 3) → email it to the client with one instruction: "paste before `</body>`" — or offer to do it on a 10-minute call.
5. Test their robot: "What are your hours?" and "Book for Friday 10am".

Day 3 — Set up monitoring (20 min)

Free uptime checks so you know before your client does:

- **No keep-alive needed:** Cloudflare Workers don't sleep, so the old 30-minute keep-alive workflow is gone. Monitor request counts + Gemini quota instead (Cloudflare dashboard → Workers → `chatbot-api`).

- **Uptime Kuma** (free, or a hosted instance): add a monitor for `https://chatbot-api.YOUR-SUBDOMAIN.workers.dev/health` every 60 s, with notifications to Telegram/email. You'll get the ping when it's down. Workers don't sleep, so any outage is a real one.
- **Email quota**: in Apps Script, run `checkQuota()` occasionally — it prints how many emails you have left today.

Day 4 — Launch checklist (30 min)

☐ Backend health URL returns `healthy` from a normal browser (not just curl)

☐ Widget loads on the client's site, right color, right greeting

☐ "What are your hours?" answers from their PDF correctly

☐ "Book for Friday 10am" books and emails arrive (customer + business)

☐ Double-booking refused (book same slot twice)

☐ Admin dashboard shows the conversation and appointment you just created

☐ Mobile test on the client's site — button, chat, slots all work

☐ Client has your phone number and knows what the robot can/can't do

Day 5 — Send your first invoice

Keep it simple. A one-page invoice with: your business name, date, client name, service line ("AI website chatbot + appointment system — monthly"), amount, and payment link (e.g. a free **Stripe payment link** or PayPal.me — both are free to set up and take a small cut per payment).

INVOICE #001 – Bob's Plumbing

Date: [today]

AI chatbot + appointment booking (setup)	\$0	(you choose: free setup for pilot)
--	-----	------------------------------------

Monthly subscription (incl. support)	\$79
--------------------------------------	------

TOTAL	\$79
-------	------

Due: within 14 days · Pay: [your payment link]

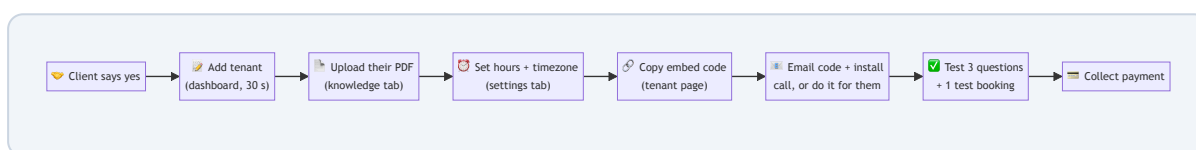
Ask for the first month up front. A client who pays is a client who's real.

WEEK 4 DONE = YOU'RE LIVE. One paying client, a monitored system, and a repeatable 5-minute onboarding recipe. That recipe is now your business.

8. Running the Business

This is the chapter the technical guides don't have: how the thing makes money and keeps you sane.

8.1 Client onboarding — the 5-minute recipe



The whole loop takes ~5 minutes once the PDF is ready. Rehearse it until it's boring — boring is fast, fast is profitable.

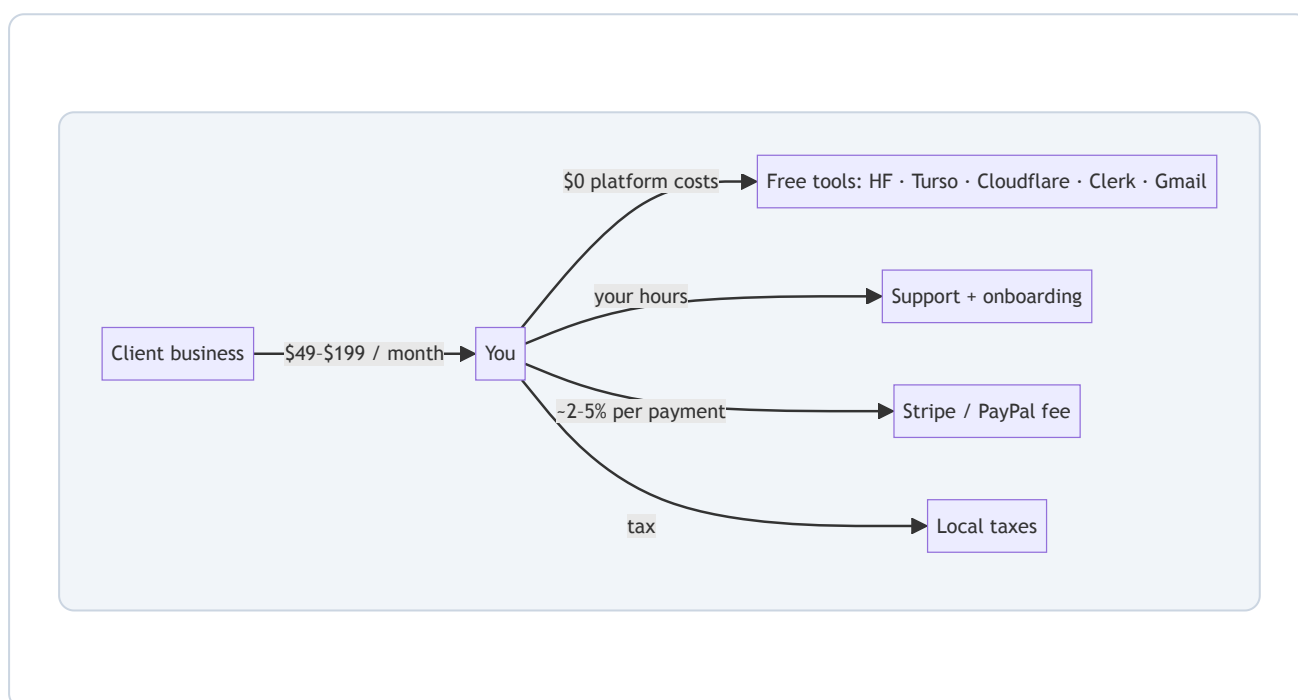
8.2 Pricing models that work for local businesses

Model	Example	Good for	Watch out for
Flat monthly	\$79/mo, everything included	Simplicity; easy to compare	You eat heavy users' costs
Tiered	Starter \$49 (chat only) / Pro \$99 (chat + bookings) / Plus \$149 (chat + bookings + email campaigns)	Up-sell path; matches effort	More decisions for you
Setup fee + monthly	\$150 setup + \$59/mo	Pays you for onboarding time	Scares off tiny businesses

Per-booking commission	\$1–\$2 per appointment	Zero-risk pitch; aligns incentives	Hard to predict revenue; clients may resent it
-------------------------------	-------------------------	------------------------------------	--

Practical advice: start with **setup fee + monthly** for your first few clients (your onboarding is still slow), then move to flat monthly once you're fast.

8.3 Where the money flows



8.4 Support boundaries (protect your sanity)

- **In scope:** robot not answering, bookings not arriving, widget missing, PDF updates (first 2/month free, then \$25 each or teach them the dashboard).
- **Out of scope:** their website, their email server, their domain, design work. Say "I'm happy to point you to someone" — never silently absorb it.
- **Response promise:** "within 24h, usually much faster" — never "24/7" unless you mean it.
- **Status page:** the health URL + Uptime Kuma means you can answer "is it down?" with a link instead of a panic.

8.5 Scaling triggers

Trigger	Move
5 clients and onboarding eats weekends	Raise setup fee; batch onboarding days
Chat latency > 5 s or > 3 chats at once	Upgrade the Space CPU/GPU (from ~\$0.60/hr) or split clients across two Spaces
Gmail quota > 80 emails/day	Google Workspace (\$6/user/mo → 1,500/day)
Turso reads near 1 B/month	Add caching or upgrade (\$29/mo)
You want to stop doing support	Hire a part-time VA for the ticket queue + monthly PDF updates

8.6 Hiring your first VA

At ~15 clients the support load becomes real. Hire before you're drowning: a part-time virtual assistant (10 h/week) who is taught your 5-minute recipe — add tenants, upload PDFs, answer client messages from a script, run the monthly checklists. You keep: pricing, sales, and anything touching money. The recipe in this guide is the training manual; write it down once and you can hand it over.

ONE SALE ENGINE THAT COSTS NOTHING: screenshots + one sentence. After every successful install, ask the client: "mind if I show this to other local businesses?" Collect one photo/quote per client. That's your entire marketing deck.

9. Troubleshooting — Top 20

"If X happens, do Y." These cover everything a non-technical operator is likely to hit. When in doubt, the health check tells you which half of the world is broken:

Open: `https://chatbot-api.YOUR-SUBDOMAIN.workers.dev/widget/health`
→ "healthy" = your system is fine, the problem is on the site/browser side
→ error/timeout = your system is waking up or down, check the Space dashboard

#	If this happens...	...do this
1	Widget button doesn't appear on the client's site	Check the script tag is exactly <code><script src="https://chatbot-widget.pages.dev/widget.js" data-tenant="their-slug"></script></code> , before <code></body></code> . Open the site, press F12 → Console — any red errors point to the cause (CSP, wrong slug, blocked script).
2	Button appears, chat says "Failed to fetch"	The widget's server calls are being blocked — usually the site's security policy (CSP) or the Space is asleep. Wait 30 s and retry; check the site's CSP needs the Space origin allowed.
3	First message takes 30+ seconds	The Space was asleep (free tier). It wakes itself. Tell clients this happens once after long quiet periods; the keep-alive pinger prevents it in practice.
4	Bot gives wrong answers	It only knows what's in the PDF. Re-upload a better PDF (clearer, correct facts), then re-test. Wrong hours → re-run the hours extraction or fix them in Settings.
5	Bot says "I don't have that information" for something you uploaded	The question's meaning didn't match any chunk closely enough. Rephrase the PDF section to use the words customers use, or upload an FAQ with the exact Q&A.

6	No appointment slots ever show	Business hours are missing/closed for those days. Check tenant → Settings → hours + timezone. Closed days (empty open/close) show nothing — that's correct behavior.
7	Slots show but booking says "no longer available"	Someone else booked it milliseconds before (or it's in the past). Refresh — that's the conflict check protecting the calendar. Test by booking the same slot twice yourself.
8	Times are off by hours	Timezone mismatch. The tenant's timezone must match the business's location (Settings tab). Check it before blaming the calendar.
9	Booking succeeds but no email arrives	Check your Gmail Sent folder first (all sends go through your Gmail). Then run <code>python scripts/test_gas_email.py --to your@email.com</code> in the Space terminal. If that fails, re-deploy the Apps Script and run <code>testEmail()</code> to re-authorize. Remember the 100/day limit.
10	Emails land in spam	Recipients click "not spam". Gmail→Gmail is usually fine. Keep email subject lines normal (no "FREE!!!").
11	Client says chat history disappeared	A refresh with private/incognito browsing loses the session (localStorage). Normal — a new thread starts. Not a data loss bug.
12	Admin login loops	Clerk doesn't know your admin URL yet. Add <code>chatbot-admin.pages.dev</code> to Clerk → Domains and allowed redirect URLs. Also confirm your email is in the allowlist.
13	Logged in but dashboard errors / empty lists	Usually the admin page can't reach the backend (network) or the auth token isn't attached. Check the browser console for 401/403 and verify your email = <code>ADMIN_EMAIL</code> secret.
14	"Tenant not found" when testing	Slug typo (case matters) or the tenant was created in a different database than the one the Space uses. Check Turso: <code>SELECT slug FROM tenants; .</code>
15	Uploaded PDF does nothing	The script must print "Created N chunks". If it errors, the file path was wrong or the tenant slug was wrong. If it succeeds but the bot still doesn't know things, the PDF was scanned images (no text) — convert it to real text first.

16	Everything was working, now the health URL is down	Space crashed or hit a quota. Check the Space page (it shows a red/green pulse), open the terminal, look at <code>tail /var/log/supervisor/fastapi_err.log</code> . Restart the Space from the dashboard if needed. (Both <code>.../health</code> and <code>.../widget/health</code> are served by the app.)
17	Chat works, but answers take 10 s even when awake	CPU inference is just slow on the free tier. Normal. Keep answers short (<code>max_tokens</code> caps already help), and consider a paid GPU upgrade when clients complain.
18	New client's widget shows the OLD client's colors/greeting	You copied the embed code with the wrong <code>data-tenant</code> slug. Check the slug on the client's page matches their tenant exactly.
19	Bot books appointments for closed days	Business hours JSON is missing or the tenant timezone is wrong. Set both in Settings. If it persists, re-run the hours extraction.
20	You changed something, deployed, and nothing changed	Deploys are async. Check the Actions tab for the workflow run to finish (green). Then hard-refresh the page (Cmd+Shift+R). Then test.

WHEN TO ASK FOR HELP: if the health URL is healthy but you're stuck for 30+ minutes, or you see errors you don't understand in a log — that's the moment to open the deep-dive guide (its Chapter 12 has root-cause analysis for every one of these) or ask a developer friend with this repo open. Don't Google-fix production by trial and error.

10. Quick Reference Cards

Print this chapter. Stick it next to your monitor.

10.1 All commands (against the Worker API)

```
# Set once: your Worker URL + bootstrap token (see DEPLOY.md §5)
API="https://chatbot-api.YOUR-SUBDOMAIN.workers.dev"
TOKEN="your-admin-token"

# One-time: load the database schema (safe to re-run)
turso db shell chatbot-db < worker/db/schema.sql

# Create a client (tenant)
curl -X POST "$API/admin/tenants" -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"name":"Client Name","slug":"client-slug"}'

# Upload their knowledge (text; PDFs are extracted on your machine first)
curl -X POST "$API/admin/tenants/TENANT_ID/knowledge/text" \
  -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
  -d '{"source_id":"faq-1","source_type":"txt","content":"..."}'

# Test the widget endpoints (no auth needed)
curl "$API/widget/config/client-slug"
curl -N -X POST "$API/widget/chat/client-slug" -H "Content-Type:
application/json" \
  -d '{"message":"What are your hours?"}'

# (The old python scripts/ are legacy – they talk directly to the DB and are
# being ported to the Worker API in Phase 4.)
```

10.2 The 11 GitHub secrets

TURSO_DATABASE_URL · TURSO_AUTH_TOKEN · GEMINI_API_KEY ·
CLERK_PUBLISHABLE_KEY · GAS_WEBAPP_URL (Phase 2) · ADMIN_EMAILS ·
ADMIN_BOOTSTRAP_TOKEN (optional, onboarding only) · CLOUDFLARE_API_TOKEN ·
CLOUDFLARE_ACCOUNT_ID — no Hugging Face secrets anymore

10.3 Your URLs (fill in)

What	URL
Backend health	<code>https://chatbot-api.YOUR-SUBDOMAIN.workers.dev/health</code>
Widget file	<code>https://chatbot-widget.pages.dev/widget.js</code>
Admin dashboard	<code>https://chatbot-admin.pages.dev</code>
Client install line	<code><script src="https://chatbot-widget.pages.dev/widget.js" data-tenant="client-slug" data-api-url="https://chatbot-api.YOUR-SUBDOMAIN.workers.dev"></script></code>
Database (Turso)	<code>https://turso.tech</code> → your DB

10.4 Weekly checklist (10 min)

- ☐ Health URL is healthy
- ☐ No red Actions runs (Worker + Pages deploys)
- ☐ Gmail quota check (`checkQuota()` in Apps Script)
- ☐ Turso dashboard — reads/storage not spiking
- ☐ One test booking per active client (Friday 10 am slot — book it, cancel it, done)

10.5 Monthly checklist (30 min)

- ☐ Review admin Analytics — who's using it, who isn't

☐ Invoice every client (same day every month)

☐ Ask each client for one piece of feedback + one testimonial

☐ Update PDFs that clients mentioned (first 2 free)

☐ Rotate the Gemini key + Cloudflare token if anything looked suspicious

10.6 Glossary — every word you need

Word	Plain meaning
Tenant	A client business — one row in the database, one robot personality
Slug	The robot's short address, used in install code (<code>bobs-plumbing</code>)
Widget	The floating chat button + window on a website
Backend	The invisible computer that does the thinking (the AI brain's server)
Frontend	The visible part — the widget and the dashboard
API	The rules computers use to talk to each other over the internet
Database	The filing cabinet that stores everything
Deploy	Push your newest code so it goes live (GitHub does this automatically)
Secrets	Private passwords/tokens stored in GitHub, never in code
RAG	"Retrieval-Augmented Generation" — the robot searches your PDF, then answers from what it found (that's why it doesn't hallucinate)
Embedding	A "meaning fingerprint" — 768 numbers that capture what a piece of text means
SSE	"Server-Sent Events" — how the answer streams in as you watch, like ChatGPT

Intent	What the visitor wants (book, ask, cancel...) — the robot classifies this first
Buffer	Extra minutes the robot keeps free around each booking (default 15)
Rate limit	A speed bump that stops anyone spamming the robot
CORS / CSP	Browser security rules that sometimes block the widget — the two acronyms that cause most "it worked on my machine" moments
VA	Virtual assistant — your first hire when there are too many clients

YOU'RE READY. Four weeks from now you'll have a live product, a paying client, and a recipe. The deep-dive guide is there when you want to understand *why* any of this works — start at Chapter 1 and read as far as curiosity takes you.

Appendix — Migration Roadmap: Phases 1–4 in Detail

Why this appendix exists: the original codebase deployed its backend as a Docker Space on Hugging Face. In July 2026 Hugging Face moved Docker (and Gradio) Spaces behind a paid PRO subscription, leaving only Static Spaces free. The project was re-planned around a **\$0 Cloudflare stack** and is being ported in phases (tracked in `hf-docker-exit-spec.md`). This guide describes the Phase-1 build; the phases below show what ships next.

Phase	What ships	Where (code)	Done when
1 — Chat + RAG  <i>implemented</i>	Widget chat (SSE), RAG answers from per-tenant knowledge, conversation history, admin-lite tenant + knowledge ingest, GitHub Actions deploys	<code>worker/</code> (config, llm, rag, chat, clerk, admin)	Widget on a test page answers from knowledge; <code>/health</code> green
2 — Appointments + email	Availability slots, booking/cancel in the chat, business-hours parsing, Google Apps Script email confirmations, <code>email_pending</code> retry UX on Gmail quota	<code>worker/src</code> ports of <code>appointments.py</code> + <code>email.py</code> ; <code>gas-email/</code>	"Book Tuesday 2pm" books + emails both sides

3 — Full admin dashboard	Tenant CRUD, analytics, conversations, appointments, usage, PDF upload (browser-side pdfjs-dist extraction), Clerk-only auth (bootstrap token removed)	Admin APIs in <code>worker/src/admin.ts</code> ; <code>admin-dashboard/</code> wired via <code>VITE_API_URL</code>	Admin UI matches the old README feature list
4 — Cleanup & docs	Remaining docs/guides rewritten, legacy <code>scripts/</code> ported to Worker API calls, old FastAPI <code>backend/</code> removed once parity is proven	Repo root	Repo has no Hugging Face / Docker remnants

Key decisions locked in the spec

- **LLM:** Gemini free tier (flash-class chat model set via `CHAT_MODEL` ; `gemini-embedding-001` @ 768 dims so the vec0 `FLOAT[768]` vector table needs no migration). Groq / Workers-AI are optional failovers. Free-tier limits are volatile — verify in AI Studio and treat as config.
- **Database:** Turso stays (sqlite-vec is built in). Schema: `worker/db/schema.sql` .
- **PDFs:** not parsed in the Worker (free-plan CPU limit) — extracted in the browser or CLI, then uploaded as text (§5.6 of the spec).
- **Auth:** Clerk JWKS for `/admin/*` ; optional `ADMIN_BOOTSTRAP_TOKEN` only for initial onboarding, deleted in Phase 3.
- **Deploys:** GitHub Actions → `deploy-worker.yml` (Worker) + Pages workflows (widget/admin).

Reading the rest of this guide

Chapters 1–4 (big picture, free tools, how it works, Week 1) are current. Chapters about the backend internals describe the reference implementation in `backend/` that the Worker is ported from. Chapters 5–7+ (appointments, emails, PDF uploads, the full

dashboard) are the Phase 2–4 roadmap — the endpoints exist in the old backend and are being ported in order. Where a command says `https://...hf.space/...`, substitute `https://chatbot-api.<subdomain>.workers.dev/...`. The live build steps always live in `DEPLOY.md`.

Practical Guide — Multi-Tenant AI Chatbot SaaS project | All tools free, no credit card required